Spark on Clusters

Spark Cluster Options

- The simple Spark cluster is shown as an alternative example – we don't use this setup as we have a Hadoop cluster running and we need a distributed fiele system such as HDFS to manage our data.
- Standalone – a simple cluster manager included with Spark that makes it easy to set up a cluster.
- Apache Mesos – a general cluster manager that can also run Hadoop MapReduce and service applications.
- Hadoop YARN – the resource manager in Hadoop.
- Kubernetes – an open-source system for automating deployment, scaling, and management of containerized applications.

Spark on the cscluster

- Spark is already installed and configured on the master and all of the worker nodes.
- Once we tunnel in via SSH (for port 8080), the Spark master's web UI is at http://localhost:8080, where we can also find the master's URL to use for our Spark programs.

```
ssh -L 8080:localhost:8080 cscluster00
```

- The Spark master's URL is spark://cscluster00.boisestate.edu:7077. We can pass it as the "master" argument to SparkContext or to spark-submit.
- Now we can run a Spark program on the cluster as follows (we are redirecting standard stream 1 – otherwise all of the words will come out on the console!). Note that we are assuming a shared local filesystem so all Spark nodes can access the data.

```
spark-submit --class "WordCount" --master spark://cscluster00.
    boisestate.edu:7077  wordcount-basic.jar input  2> log 1>
    output
```

Spark on the cscluster (2)

- A Spark program running on local host can connect with Hadoop HDFS running in pseudo-distributed mode on the same local host.
- A Spark program running on a standalone cluster needs to connect with Hadoop HDFS running on a cluster, typically the same cluster.

```
spark-submit --class "WordCount" --master spark://cscluster00.
    boisestate.edu:7077 wordcount-hdfs-save.jar hdfs://
    cscluster00:9000/user/amit/input
```

- Note that the `wordcount-hdfs-save.jar` is a modified version of the word count program that gets the input from HDFS using the following:

```
sc.textFile("hdfs://cscluster00.boisestate.edu:9000/user/amit/
    input");
```

- Note that the `wordcount-hdfs-save.jar` is a modified version of the word count program that saves the output to HDFS using the following:

```
counts.saveAsTextFile("hdfs://cscluster00.boisestate.edu:9000/
    user/amit/output");
```

Installing Spark on a Standalone Cluster (1)

- Spark needs to be installed on the master and all of the nodes. For the lab, we installed Spark on `cscluster00`. Since all nodes share the home file system, there is no need to install Spark on the nodes.
- Start the master (typically, we would use `cscluster00.boisestate.edu`)

```
[amit@cscluster00 ~]$ start-master.sh
starting org.apache.spark.deploy.master.Master, logging to
/home/amit/spark-install/spark/logs/spark-amit-org.apache.spark.deploy.
    master.Master-1-cscluster00.boisestate.edu.out
```

- Once started, the master will print out a `spark://HOST:PORT URL` for itself, which we can use to connect workers to it, or pass as the "master" argument to `SparkContext` or to `spark-submit`.

```
[amit@cscluster00 ~]$ cat /home/amit/spark-install/spark/logs/spark-amit
    -org.apache.spark.deploy.master.Master-1-cscluster00.boisestate.edu
    .out | grep spark:
19/11/13 23:48:56 INFO Master: Starting Spark master at spark://
    cscluster00.boisestate.edu:7077
```

- The master's web UI is at `http://localhost:8080`, where we can also find the master's URL.

Installing Spark on a Standalone Cluster (2)

- Start one or more workers on multiple nodes by passing in the Spark URL of the master. We need password-less ssh access.

```
[amit@cscluster00 ~]$ ssh cscluster01
Last login: Tue Nov 12 14:27:47 2019 from cscluster00.boisestate.edu

[amit@cscluster01 ~]$ start-worker.sh spark://cscluster00.boisestate.edu
    :7077

starting org.apache.spark.deploy.worker.Worker, logging to /home/amit/
    spark-install/spark/logs/spark-amit-org.apache.spark.deploy.worker.
    Worker-1-cscluster01.boisestate.edu.out
```

- Once we have started a worker, look at the master's web UI. We should see the new node listed there, along with its number of CPUs and memory (minus one gigabyte left for the OS).
- We can now run Spark jobs on the cluster. For example:

```
spark-submit --class "WordCount" --master spark://cscluster00.boisestate
    .edu:7077  wordcount-basic.jar input  2> log 1> output
```

- Finally, we can stop the master and worker daemons.

```
[amit@cscluster00 ~]$ ssh cscluster01 stop-worker.sh
stopping org.apache.spark.deploy.worker.Worker
[amit@cscluster00 ~]$ stop-master.sh
stopping org.apache.spark.deploy.master.Master
```

Installing Spark on a Standalone Cluster (3)

- We can automate the start/stop of a standalone cluster. Add the names of the worker nodes to the file ~/spark-install/spark/conf/workers. Then use the start-all.sh and stop-all.sh scripts.

```
[amit@cscluster00 ~]$ start-all.sh
starting org.apache.spark.deploy.master.Master, logging to /home/amit/
    spark-install/spark/logs/spark-amit-org.apache.spark.deploy.master.
    Master-1-cscluster00.boisestate.edu.out
cscluster03.boisestate.edu: starting org.apache.spark.deploy.worker.
    Worker, logging to /home/amit/spark-install/spark/logs/spark-amit-
    org.apache.spark.deploy.worker.Worker-1-cscluster03.boisestate.edu.
    out
cscluster01.boisestate.edu: starting org.apache.spark.deploy.worker.
    Worker, logging to /home/amit/spark-install/spark/logs/spark-amit-
    org.apache.spark.deploy.worker.Worker-1-cscluster01.boisestate.edu.
    out

[amit@cscluster00 ~]$ stop-all.sh
cscluster03.boisestate.edu: stopping org.apache.spark.deploy.worker.
    Worker
cscluster01.boisestate.edu: stopping org.apache.spark.deploy.worker.
    Worker
stopping org.apache.spark.deploy.master.Master
```

Running Spark using Hadoop YARN scheduler

- We can redirect Spark to use Hadoop YARN as the job scheduler by first setting the following environment variable.

```
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

- Then we can simply change the `-master` option for `spark-submit` to be `yarn` and viola! For example:

```
spark-submit --master yarn --class WordCount wordcount-hdfs-
    save.jar hdfs://cscluster00:9000/user/amit/input
```

- To make Hadoop YARN the default for Spark, we can do the following steps to set the default. However, thsi isn't recommended as the users can esily control client versus cluster node when they submit jobs.

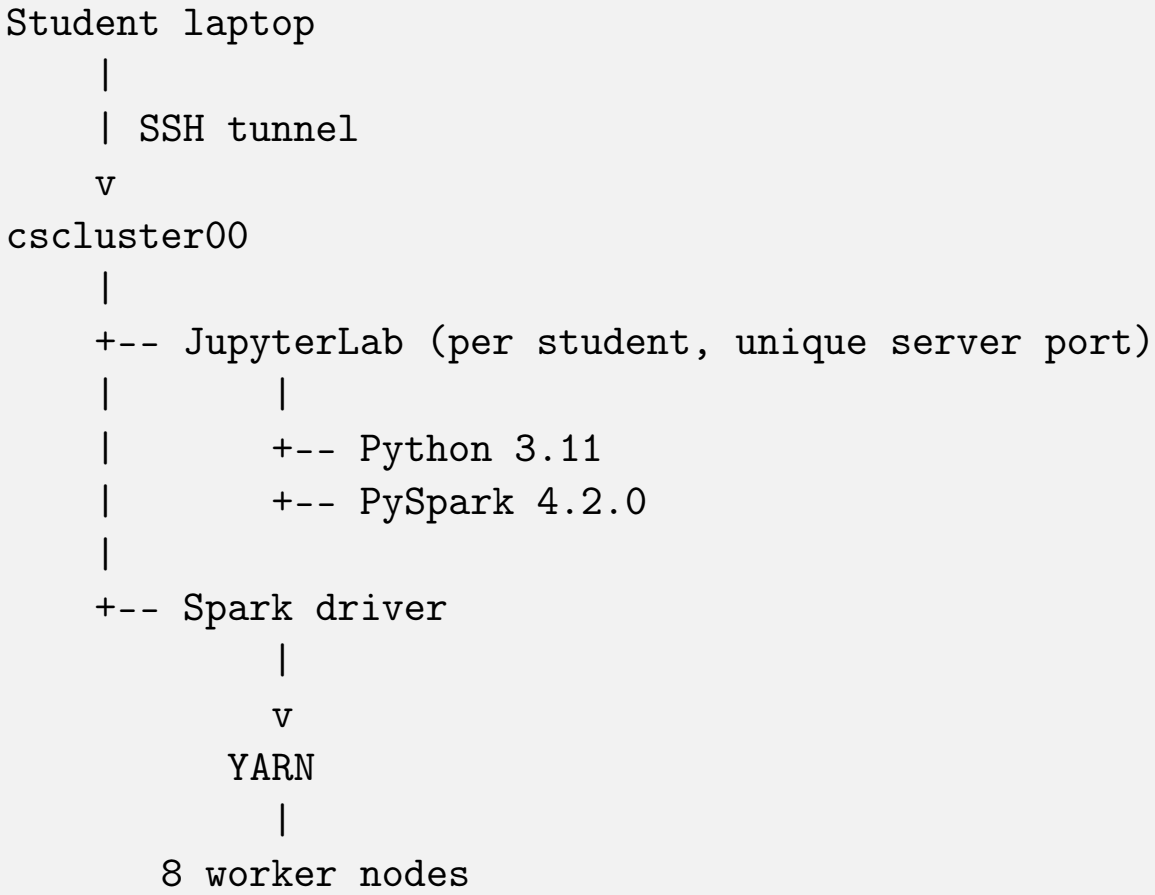
```
mv $SPARK_HOME/conf/spark-defaults.conf.template $SPARK_HOME
    /conf/spark-defaults.conf
//edit spark-defaults.conf and add the following line
spark-master yarn
```

Running Spark on a Cluster

- Spark jobs can run on spark/hadoop clusters in two modes: cluster mode and client mode.
- Recall that a Spark job consists of two parts: Spark Executors that run the actual tasks, and a Spark Driver that schedules the Executors.
  - **cluster** mode: everything runs inside the cluster. We can start a job from cscluster00 and the job will continue running even if we close the ssh session.
  - **client** mode: the Spark driver runs on a client, such as cscluster00. If the ssh session or the client is shut down, the job fails.
- Client mode is well suited for interactive jobs, but applications will fail if the client stops. For long running jobs, cluster mode is more appropriate.

Using Jupyter Lab on the Cluster

- We can run Jupyter on the cluster and access it from our laptop! The following diagram shows the architecture.



- The students/users do not install python, Jupyter, PySpark, Spark, Hadoop etc on cscluster00. The cluster administrator has already installed and configured all of these for us. The users don't even really need Python, Jupyter, PySpark, Spark, Hadoop etc on their laptops either. The users just need a web browser and an SSH client.

Using Jupyter Lab on the Cluster (2)

- Make sure you are on campus network (physically or via VPN).
- First login to cscluster00 and start up a headless jupyter server. Note the port number that is printed out.

  ```
  ssh <yourBoiseStateLogin>@cscluster00
  ```

- After logging in, start up a headless jupyter server. Note the port number that is printed out.

  ```
  start-jupyter
  ```

- Back on your laptop, start a new terminal. Then run the following command to create a secure tunnel from your laptop to the cluster. Use the port number that was printed out by the `start-jupyter` command (we add 18080 for Spark History server):

  ```
  ssh -L <YOUR_PORT>:localhost:<YOUR_PORT> \
      -L 18080:localhost:18080 \
      <yourBoiseStateLogin>@cscluster00>
  ```

- Then, open the web interface to Jupyter Lab in your browser using the following URL (where the token is shown in the output from jupyter lab when we start it as above):

  ```
  http://localhost:<YOUR_PORT>/lab?token=...
  ```